

AWS Backend Setup Progress Document

COMPLETE STEP-BY-STEP IMPLEMENTATION FLOW

STEP 1 — AWS ACCOUNT SETUP

1. Create AWS account
 2. Login using root account
 3. Enable MFA
 4. Open IAM service
 5. Create admin group
 6. Attach AdministratorAccess policy
 7. Create IAM user:
 8. backend-admin
 9. Enable console access
 10. Add user to admin group
 11. Login using IAM user
-

STEP 2 — EC2 SERVER CREATION

1. Open EC2 Dashboard
2. Launch Instance
3. Choose Ubuntu Server 22.04
4. Select t3.medium
5. Create PEM key pair
6. Open inbound ports:
 7. 22
 8. 80
 9. 443
 10. 5000
 11. 3000
12. Launch EC2
13. Copy public IP
14. SSH into server

SSH Command:

```
ssh -i business_prod_key.pem ubuntu@SERVER_IP
```

STEP 3 — SERVER ENVIRONMENT SETUP

Run:

```
sudo apt update && sudo apt upgrade -y
```

Install Node.js:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - sudo apt install nodejs -y
```

Install PM2:

```
sudo npm install -g pm2
```

Install Git:

```
sudo apt install git -y
```

Install Nginx:

```
sudo apt install nginx -y
```

Install PostgreSQL Client:

```
sudo apt install postgresql-client -y
```

STEP 4 — POSTGRESQL RDS SETUP

1. Open RDS
 2. Create Database
 3. Choose PostgreSQL
 4. Free tier
 5. DB Name: businessdb
 6. Username: postgres
 7. Set password
 8. Enable Public Access
 9. Configure security group
 10. Create database
 11. Wait until Available
-

STEP 5 — SECURITY GROUP FIXES

1. Open RDS security group
 2. Add inbound rule:
 3. PostgreSQL
 4. Port 5432
 5. Source = EC2 Security Group
 6. Save rules
-

STEP 6 — DATABASE CONNECTION

Connect:

```
psql -h businessdb.cro6208kk5re.ap-south-1.rds.amazonaws.com -U postgres -d postgres
```

Create database:

```
CREATE DATABASE businessdb;
```

Reconnect:

```
psql -h businessdb.cro6208kk5re.ap-south-1.rds.amazonaws.com -U postgres -d businessdb
```

STEP 7 — BACKEND PROJECT CREATION

```
mkdir backend cd backend
```

```
npm init -y
```

STEP 8 — INSTALL BACKEND PACKAGES

```
npm install express cors dotenv bcrypt jsonwebtoken prisma @prisma/client multer socket.io
```

```
npm install -D nodemon
```

STEP 9 — PRISMA SETUP

`npx prisma init`

Downgraded Prisma to version 5.

Configured DATABASE_URL in .env

Created Prisma schema.

Run migration:

`npx prisma migrate dev --name init`

STEP 10 — BACKEND FOLDER STRUCTURE

Created:

`src/controllers src/routes src/middleware src/services src/socket src/utils`

STEP 11 — EXPRESS SERVER SETUP

Created:

- `src/app.js`
- Prisma utility
- Auth routes
- Auth controller

Configured:

- Express
 - CORS
 - JSON parsing
 - API routing
-

STEP 12 — AUTHENTICATION SYSTEM

Implemented:

- Register API
- Login API

- JWT token generation
- Password hashing
- Role system

Roles:

- ADMIN
- OPERATOR
- INVESTOR

STEP 13 — API TESTING

Backend URL:

http://13.207.1.144:5000

Tested:

- Root endpoint
- Register API
- Login API

Postman CURL commands created.

STEP 14 — NEXT MODULES PENDING

1. JWT middleware
2. Role middleware
3. Business CRUD APIs
4. Investor management
5. Operator management
6. Orders APIs
7. Profit engine
8. Dashboard APIs
9. Printer APIs
10. Socket.IO
11. SSL setup
12. Nginx reverse proxy
13. PM2 production setup
14. Deployment scripts
15. CI/CD

AWS Backend Setup Progress Document

Project Overview

Business Management Platform with:

- Multi-business support
 - Investor management
 - Admin portal
 - Operator app
 - Investor app
 - Wireless thermal printer integration
 - Profit sharing engine
 - Real-time order updates
-

Infrastructure Completed

AWS Account Setup

Completed:

- AWS account created
- IAM admin user created
- Administrator access attached
- MFA/security recommendations reviewed

IAM User:

- Username: backend-admin
-

EC2 Server Setup

Completed:

- EC2 Ubuntu Server 22.04 created
- Instance Type: t3.medium
- SSH access configured
- PEM key generated

Security Group Ports Opened:

- 22 (SSH)

- 80 (HTTP)
- 443 (HTTPS)
- 5000 (Backend API)
- 3000 (Socket/Frontend Dev)

EC2 Public IP: 13.207.1.144

PostgreSQL Database Setup

Completed:

- PostgreSQL RDS instance created
- Public access enabled
- Security group configured
- EC2 ↔ RDS connectivity fixed
- PostgreSQL connection verified

Database:

- businessdb

RDS Endpoint: businessdb.cro6208kk5re.ap-south-1.rds.amazonaws.com

PostgreSQL Port: 5432

Database Connection Commands

Connect To PostgreSQL

```
psql -h businessdb.cro6208kk5re.ap-south-1.rds.amazonaws.com -U postgres -d postgres
```

Create Database

```
CREATE DATABASE businessdb;
```

Connect To businessdb

```
psql -h businessdb.cro6208kk5re.ap-south-1.rds.amazonaws.com -U postgres -d businessdb
```

Backend Environment Setup

Completed:

- Node.js installed
- npm installed
- PM2 installed
- Git installed
- Nginx installed
- PostgreSQL client installed

Installed Packages:

- express
 - cors
 - dotenv
 - bcrypt
 - jsonwebtoken
 - prisma
 - @prisma/client
 - multer
 - socket.io
-

Backend Folder Structure

Created:

```
backend/ ├── prisma/ ├── src/ | ├── controllers/ | ├── middleware/ | ├── routes/ | ├──  
services/ | ├── socket/ | └── utils/
```

Prisma ORM Setup

Completed:

- Prisma initialized
- Prisma schema created
- PostgreSQL datasource configured
- Prisma migration completed

Main Models Created:

- User

- Business
- Investor
- InvestorTransaction
- Operator
- Printer
- Order

Enums:

- UserRole
 - OrderStatus
-

Authentication System

Completed:

- JWT authentication
- User registration API
- User login API
- Password hashing using bcrypt
- Role-based architecture

Roles:

- ADMIN
 - OPERATOR
 - INVESTOR
-

Authentication APIs

Register API

POST /api/auth/register

Sample Body:

```
{ "name": "Admin", "mobile": "9999999999", "password": "123456", "role": "ADMIN" }
```

Login API

POST /api/auth/login

Sample Body:

```
{ "mobile": "9999999999", "password": "123456" }
```

API Server Setup

Main File:

- src/app.js

Server Port: 5000

Root Endpoint: /

Response:

```
{ "success": true, "message": "Business Backend Running" }
```

Current Working Components

Working:

- AWS EC2
 - PostgreSQL RDS
 - SSH
 - Node.js
 - Prisma
 - Express server
 - JWT authentication
 - User registration
 - Login APIs
-

Pending Modules

To Be Built:

1. JWT middleware
2. Role middleware
3. Business CRUD APIs
4. Investor APIs
5. Operator APIs

6. Orders APIs
 7. Profit engine
 8. Settlement engine
 9. Dashboard APIs
 10. Printer APIs
 11. Socket.IO realtime sync
 12. Admin portal APIs
 13. Investor wallet APIs
 14. Reports APIs
 15. PDF generation
 16. Notifications
 17. Deployment scripts
 18. SSL HTTPS setup
 19. Domain setup
 20. CI/CD pipeline
-

Important Commands

Start Backend

```
node src/app.js
```

Start Using PM2

```
pm2 start src/app.js --name business-api
```

PM2 Logs

```
pm2 logs
```

PM2 Status

```
pm2 status
```

Deployment Notes

Backend Server IP: 13.207.1.144

Backend Base URL: <http://13.207.1.144:5000>

Future Architecture

Flutter App ↓ Admin Portal ↓ Node.js APIs ↓ Socket.IO ↓ PostgreSQL RDS ↓ AWS Infrastructure

Project Goal

A scalable SaaS-ready business management ecosystem with:

- investor profit management
- business operations
- wireless printer integration
- real-time tracking
- multi-business architecture
- secure role-based access